

pandas_hands_on_learning

January 20, 2026

1 Pandas Hands-On Learning Adventure

Welcome! This notebook is all about **learning by doing**. You'll write code, change things, break things, and see what happens. That's how you really learn!

1.1 The Vibe

- **Experiment freely** - change the code and run it again
- **Make mistakes** - they're learning opportunities
- **Observe outcomes** - see what each change does
- **Have fun** - pandas is powerful and actually pretty cool!

Let's dive in!

1.2 Setup - Run This First!

Every pandas adventure starts with imports. Run this cell to get started:

```
[ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print(" Ready to rock! Pandas version:", pd.__version__)
```

1.3 Level 1: Creating Your First Data

Let's start with a **Series** - think of it as a fancy list with labels.

```
[ ]: # Create a Series from a list
s = pd.Series([1, 3, 5, np.nan, 6, 8])
print(s)
```

1.3.1 Your Turn! Experiment Time:

Try these modifications and see what happens: 1. Change the numbers in the list 2. Add more numbers 3. Remove some numbers 4. What does `np.nan` represent? Try removing it and running again

```
[ ]: # YOUR PLAYGROUND - Experiment here!
s = pd.Series([1, 3, 5, np.nan, 6, 8]) # Modify this line!
print(s)

# Try adding this line to see the data type:
# print("\nData type:", s.dtype)
```

1.4 Level 2: DataFrames - Where The Magic Happens

A **DataFrame** is like a spreadsheet or a table. It's what makes pandas powerful!

```
[ ]: # Create a date range
dates = pd.date_range('20240101', periods=6)
print("Our dates:")
print(dates)
```

```
[ ]: # Create a DataFrame with random numbers
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list('ABCD'))
print(df)
```

1.4.1 Your Turn! Play With DataFrames:

Experiments to try: 1. Change periods=6 to periods=10 - what happens? 2. Change the column names from 'ABCD' to something else like 'WXYZ' 3. Change the dimensions: try (10, 3) instead of (6, 4) (but watch out - rows and columns must match!)

```
[ ]: # YOUR PLAYGROUND
dates = pd.date_range('20240101', periods=6) # Try changing periods!
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list('ABCD')) #
    ↪Experiment here!
print(df)
```

1.4.2 Let's Create a More Interesting DataFrame!

```
[ ]: # Create a DataFrame with mixed data types
df2 = pd.DataFrame({
    'A': 1.0,
    'B': pd.Timestamp('20240102'),
    'C': pd.Series(1, index=range(4), dtype='float32'),
    'D': np.array([3] * 4, dtype='int32'),
    'E': 'pandas is cool'
})
print(df2)
```

1.4.3 Experiment Challenge:

Try modifying: 1. Change the text in column 'E' to something else 2. Change the number 3 in column 'D' to a different number 3. Change the date in column 'B' 4. Add a new column 'F' with your own data!

```
[ ]: # YOUR PLAYGROUND - Get creative!
df2 = pd.DataFrame({
    'A': 1.0,
    'B': pd.Timestamp('20240102'),
    'C': pd.Series(1, index=range(4), dtype='float32'),
    'D': np.array([3] * 4, dtype='int32'),
    'E': 'pandas is cool'
})
print(df2)
print("\nData types:")
print(df2.dtypes)
```

1.5 Level 3: Viewing Your Data

Let's learn how to peek at our data efficiently!

```
[ ]: # First, let's create a bigger dataset to work with
dates = pd.date_range('20240101', periods=12)
df = pd.DataFrame(np.random.randn(12, 4), index=dates, columns=['A', 'B', 'C', 'D'])
print("Full DataFrame:")
print(df)
```

```
[ ]: # View the first few rows
print("First 5 rows:")
print(df.head())
```

```
[ ]: # View the last few rows
print("Last 3 rows:")
print(df.tail(3))
```

1.5.1 Experiment Time:

Try these: 1. Change head() to head(3) or head(7) - what changes? 2. Try tail(1) or tail(10) - observe the results 3. What happens if you ask for more rows than exist?

```
[ ]: # YOUR PLAYGROUND
print(df.head()) # Experiment with different numbers!
print("\n")
print(df.tail()) # Try different numbers here too!
```

1.5.2 Quick Stats About Your Data

```
[ ]: # Get quick statistics
print("Statistical Summary:")
print(df.describe())
```

```
[ ]: # Check the shape (rows, columns)
print("Shape (rows, columns):", df.shape)
print("Number of rows:", len(df))
print("Column names:", df.columns.tolist())
```

1.6 Level 4: Selecting Data - The Fun Part!

This is where pandas gets really powerful. Let's learn how to grab exactly the data we want.

```
[ ]: # Select a single column
print("Just column A:")
print(df['A'])
```

```
[ ]: # Select multiple columns
print("Columns A and C:")
print(df[['A', 'C']])
```

1.6.1 Your Turn - Column Selection:

Experiments: 1. Select different single columns (B, C, or D) 2. Try selecting 3 columns at once 3. What happens if you try to select a column that doesn't exist?

```
[ ]: # YOUR PLAYGROUND
print(df['A']) # Try different columns!
print("\n")
print(df[['A', 'C']]) # Select different combinations!
```

1.6.2 Slicing Rows - Like Slicing Bread!

```
[ ]: # Select rows by position (slicing)
print("Rows 0 to 3:")
print(df[0:3])
```

```
[ ]: # Select rows by date range
print("Rows from 2024-01-02 to 2024-01-04:")
print(df['20240102':'20240104'])
```

1.6.3 Experiment With Slicing:

Try these: 1. Change [0:3] to [5:10] or [2:7] - observe what you get 2. Try different date ranges 3. What happens with [0:100]? (hint: more than we have!)

```
[ ]: # YOUR PLAYGROUND
print(df[0:3]) # Change the slice!
print("\n")
print(df['20240102':'20240104']) # Try different dates!
```

1.6.4 Precision Selection with .loc and .iloc

```
[ ]: # .loc uses labels
print("Using .loc (label-based):")
print(df.loc[dates[0]])
```

```
[ ]: # .iloc uses integer positions
print("Using .iloc (position-based):")
print(df.iloc[3]) # Gets the 4th row (0-indexed)
```

```
[ ]: # Selecting specific rows and columns with .loc
print("Specific rows and columns with .loc:")
print(df.loc[dates[0]:dates[3], ['A', 'B']])
```

```
[ ]: # Selecting by position with .iloc
print("Using .iloc with row and column positions:")
print(df.iloc[3:5, 0:2]) # Rows 3-4, columns 0-1
```

1.6.5 Master Selection Challenge:

Your experiments: 1. Use .iloc[0] to get the first row, then try .iloc[-1] for the last row 2. Try .iloc[2:8, 1:3] - what do you get? 3. Use .loc to select rows 2-6 and columns B and D 4. What's the difference between .loc and .iloc?

```
[ ]: # YOUR PLAYGROUND - Get specific!
print(df.iloc[3]) # Experiment with positions!
print("\n")
print(df.loc[dates[0]:dates[3], ['A', 'B']]) # Try different combinations!
```

1.7 Level 5: Boolean Indexing - Finding What You Want

This is like asking questions about your data: "Show me all the rows where column A is positive!"

```
[ ]: # Select rows where column A is positive
print("Rows where A > 0:")
print(df[df['A'] > 0])
```

```
[ ]: # Select all positive values (negatives become NaN)
print("All positive values (negatives become NaN):")
print(df[df > 0])
```

1.7.1 Boolean Magic Experiments:

Try these conditions: 1. `df[df['A'] < 0]` - what do you get? 2. `df[df['B'] > 0.5]` - experiment with different numbers 3. `df[df['A'] > df['B']]` - comparing columns! 4. Multiple conditions: `df[(df['A'] > 0) & (df['B'] > 0)]`

```
[ ]: # YOUR PLAYGROUND - Ask your data questions!
print("My condition:")
print(df[df['A'] > 0]) # Change this condition!

# Try multiple conditions:
# print(df[(df['A'] > 0) & (df['B'] > 0)])
```

1.8 Level 6: Basic Operations & Stats

Let's do some math with our data!

```
[ ]: # Calculate mean for each column
print("Mean of each column:")
print(df.mean())
```

```
[ ]: # Calculate mean for each row
print("Mean of each row:")
print(df.mean(axis=1))
```

```
[ ]: # More statistics!
print("Sum of each column:")
print(df.sum())
print("\nStandard deviation:")
print(df.std())
print("\nMax values:")
print(df.max())
```

1.8.1 Stats Experiments:

Try these methods: 1. `.min()` - find minimum values 2. `.median()` - find median values 3. `.std()` - standard deviation 4. `.count()` - count non-null values 5. What happens when you add `axis=1` to any of these?

```
[ ]: # YOUR PLAYGROUND - Calculate statistics!
print(df.mean()) # Try different methods!
print("\n")
```

```
print(df.mean(axis=1)) # Experiment with axis!
```

1.8.2 Let's Do Some Math With Our Data

```
[ ]: # Add a number to all values
print("Original:")
print(df.head(3))
print("\nAdding 10 to everything:")
print((df + 10).head(3))
```

```
[ ]: # Multiply all values
print("Multiplying by 2:")
print((df * 2).head(3))
```

1.8.3 Math Experiments:

Try these: 1. Subtract 5 from all values: `df - 5` 2. Divide by 2: `df / 2` 3. Square all values: `df ** 2` 4. Get absolute values: `df.abs()`

```
[ ]: # YOUR PLAYGROUND - Do math!
print((df + 10).head()) # Try different operations!
```

1.9 Level 7: Working with Missing Data

Real data is messy! Let's learn to handle missing values.

```
[ ]: # Create a DataFrame with some missing values
df_missing = df.copy()
df_missing.iloc[0:2, 1] = np.nan # Add some NaN values
df_missing.iloc[3:5, 2] = np.nan
print("DataFrame with missing values:")
print(df_missing.head(7))
```

```
[ ]: # Drop rows with any missing values
print("After dropping rows with NaN:")
print(df_missing.dropna())
```

```
[ ]: # Fill missing values with a number
print("After filling NaN with 0:")
print(df_missing.fillna(0).head(7))
```

```
[ ]: # Fill with the mean of each column
print("After filling with column means:")
print(df_missing.fillna(df_missing.mean()).head(7))
```

1.9.1 Missing Data Experiments:

Try these: 1. Fill with different values: `fillna(999)` or `fillna(-1)` 2. Try `fillna(method='ffill')` - forward fill 3. Try `fillna(method='bfill')` - backward fill 4. Use `isna()` to see where the NaN values are

```
[ ]: # YOUR PLAYGROUND - Handle missing data!
print(df_missing.fillna(0).head(7)) # Try different strategies!

# Uncomment to see where NaN values are:
# print("\nNaN locations:")
# print(df_missing.isna().head(7))
```

1.10 Level 8: Grouping & Aggregating - Super Powerful!

This is where pandas really shines. Let's create some realistic data first.

```
[ ]: # Create a sales dataset
sales_data = pd.DataFrame({
    'product': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B'],
    'region': ['North', 'North', 'South', 'South', 'North', 'North', 'South', 'South'],
    'sales': [100, 150, 200, 90, 110, 170, 180, 95],
    'quantity': [10, 12, 20, 8, 11, 15, 18, 9]
})
print("Sales Data:")
print(sales_data)
```

```
[ ]: # Group by product and calculate total sales
print("Total sales by product:")
print(sales_data.groupby('product')['sales'].sum())
```

```
[ ]: # Group by region and calculate mean
print("Average sales by region:")
print(sales_data.groupby('region')['sales'].mean())
```

```
[ ]: # Multiple aggregations at once!
print("Multiple statistics by product:")
print(sales_data.groupby('product')['sales'].agg(['sum', 'mean', 'count', 'max']))
```

1.10.1 Grouping Experiments:

Try these: 1. Group by 'region' instead of 'product' 2. Try different aggregations: `.max()`, `.min()`, `.count()` 3. Group by multiple columns: `groupby(['product', 'region'])` 4. Aggregate multiple columns: `groupby('product')[['sales', 'quantity']].sum()`

```
[ ]: # YOUR PLAYGROUND - Group and aggregate!
print(sales_data.groupby('product')['sales'].sum()) # Try different groupings!

# Try this advanced one:
# print(sales_data.groupby(['product', 'region'])['sales'].mean())
```

1.11 Level 9: Merging & Joining DataFrames

Combining data from different sources - a crucial skill!

```
[ ]: # Create two DataFrames to merge
df1 = pd.DataFrame({
    'employee': ['Alice', 'Bob', 'Charlie'],
    'department': ['Sales', 'IT', 'Marketing']
})

df2 = pd.DataFrame({
    'employee': ['Alice', 'Bob', 'Charlie'],
    'salary': [70000, 80000, 65000]
})

print("DataFrame 1:")
print(df1)
print("\nDataFrame 2:")
print(df2)
```

```
[ ]: # Merge the DataFrames
merged = pd.merge(df1, df2, on='employee')
print("Merged DataFrame:")
print(merged)
```

```
[ ]: # Concatenate DataFrames vertically (stack them)
df3 = pd.DataFrame({
    'A': ['A4', 'A5', 'A6'],
    'B': ['B4', 'B5', 'B6']
})

df4 = pd.DataFrame({
    'A': ['A7', 'A8', 'A9'],
    'B': ['B7', 'B8', 'B9']
})

print("Concatenated vertically:")
print(pd.concat([df3, df4]))
```

1.11.1 Merging Experiments:

Try these: 1. Add a new employee to one DataFrame but not the other - what happens when you merge? 2. Try different merge types: `pd.merge(df1, df2, on='employee', how='left')` 3. Use `how='right'`, `how='inner'`, or `how='outer'` - observe the differences! 4. Concatenate horizontally: `pd.concat([df3, df4], axis=1)`

```
[ ]: # YOUR PLAYGROUND - Combine data!
merged = pd.merge(df1, df2, on='employee') # Try different merge types!
print(merged)

# Try this:
# print(pd.concat([df3, df4], axis=1))
```

1.12 Level 10: String Operations

Working with text data is super common. Let's see how!

```
[ ]: # Create a Series with strings
s = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca', np.nan, 'CABA', 'dog', 'cat'])
print("Original:")
print(s)
```

```
[ ]: # Convert to lowercase
print("Lowercase:")
print(s.str.lower())
```

```
[ ]: # Convert to uppercase
print("Uppercase:")
print(s.str.upper())
```

```
[ ]: # String length
print("Length of each string:")
print(s.str.len())
```

```
[ ]: # Check if string contains something
print("Contains 'a':")
print(s.str.contains('a', na=False))
```

1.12.1 String Operation Experiments:

Try these: 1. `s.str.capitalize()` - capitalize first letter 2. `s.str.replace('a', 'X')` - replace characters 3. `s.str.startswith('A')` - check if starts with a letter 4. `s.str.strip()` - remove whitespace 5. Change your search in `.contains()` to look for different patterns

```
[ ]: # YOUR PLAYGROUND - Manipulate strings!
print(s.str.lower()) # Try different string operations!
```

```
# Try these:
# print(s.str.replace('a', '!'))
# print(s.str.startswith('C'))
```

1.13 Level 11: Pivot Tables - Excel Users Will Love This!

Remember pivot tables from Excel? Pandas can do them even better!

```
[ ]: # Create a more complex dataset
df = pd.DataFrame({
    'A': ['one', 'one', 'two', 'three'] * 3,
    'B': ['A', 'B', 'C'] * 4,
    'C': ['foo', 'foo', 'foo', 'bar', 'bar', 'bar'] * 2,
    'D': np.random.randn(12),
    'E': np.random.randn(12)
})
print("Dataset:")
print(df)
```

```
[ ]: # Create a pivot table
print("Pivot Table:")
pivot = pd.pivot_table(df, values='D', index=['A', 'B'], columns=['C'])
print(pivot)
```

1.13.1 Pivot Table Experiments:

Try these: 1. Change values='D' to values='E' 2. Change the index to ['C'] 3. Try aggfunc='mean' or aggfunc='count' 4. Add margins=True to see totals

```
[ ]: # YOUR PLAYGROUND - Create pivot tables!
pivot = pd.pivot_table(df, values='D', index=['A', 'B'], columns=['C']) #
    ↪Experiment!
print(pivot)

# Try this:
# pivot = pd.pivot_table(df, values='D', index=['A'], columns=['C'],
    ↪aggfunc='count')
# print(pivot)
```

1.14 Level 12: Visualization - See Your Data!

A picture is worth a thousand rows of data!

```
[ ]: # Create time series data
ts = pd.Series(np.random.randn(1000), index=pd.date_range('1/1/2024',
    ↪periods=1000))
ts = ts.cumsum() # Cumulative sum makes it more interesting

# Plot it!
ts.plot(figsize=(12, 4))
plt.title('Random Walk Over Time')
plt.ylabel('Value')
plt.xlabel('Date')
plt.show()
```

```
[ ]: # Bar plot of our sales data
sales_summary = sales_data.groupby('product')['sales'].sum()
sales_summary.plot(kind='bar', color=['skyblue', 'coral'])
plt.title('Total Sales by Product')
plt.ylabel('Sales')
plt.xlabel('Product')
plt.xticks(rotation=0)
plt.show()
```

```
[ ]: # Multiple columns
dates = pd.date_range('20240101', periods=30)
df_plot = pd.DataFrame(np.random.randn(30, 4).cumsum(axis=0),
    index=dates,
    columns=['A', 'B', 'C', 'D'])
df_plot.plot(figsize=(12, 5))
plt.title('Multiple Time Series')
plt.legend(loc='best')
plt.show()
```

1.14.1 Visualization Experiments:

Try different plot types: 1. kind='line' - line plot (default) 2. kind='bar' - bar chart 3. kind='hist' - histogram 4. kind='box' - box plot 5. kind='scatter' - scatter plot (needs x and y) 6. Change colors: color='red' or color=['red', 'blue', 'green'] 7. Change figure size: figsize=(15, 6)

```
[ ]: # YOUR PLAYGROUND - Make it visual!
df_plot.plot(figsize=(12, 5)) # Try different plot types and styles!
plt.title('Your Custom Plot Title') # Customize this!
plt.show()

# Try a histogram:
# df_plot['A'].plot(kind='hist', bins=20)
# plt.title('Histogram of Column A')
# plt.show()
```

1.15 Bonus Level: Reading & Writing Files

In the real world, you'll often work with files. Let's practice!

```
[ ]: # Create some sample data
sample_data = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'San Francisco', 'Chicago', 'Boston'],
    'Salary': [70000, 80000, 75000, 72000]
})
print("Sample data:")
print(sample_data)
```

```
[ ]: # Save to CSV
sample_data.to_csv('my_data.csv', index=False)
print(" Saved to my_data.csv")

# Read it back
df_from_csv = pd.read_csv('my_data.csv')
print("\nRead from CSV:")
print(df_from_csv)
```

```
[ ]: # Save to Excel (requires openpyxl or xlsxwriter)
try:
    sample_data.to_excel('my_data.xlsx', index=False, sheet_name='Employees')
    print(" Saved to my_data.xlsx")

    # Read it back
    df_from_excel = pd.read_excel('my_data.xlsx')
    print("\nRead from Excel:")
    print(df_from_excel)
except ImportError:
    print(" Excel support not installed. Install with: pip install openpyxl")
```

1.15.1 File Operations Experiments:

Try these: 1. Modify the `sample_data` and save it again 2. Read the CSV with `pd.read_csv('my_data.csv', nrows=2)` to read only first 2 rows 3. Try saving with compression: `to_csv('data.csv.gz', compression='gzip')` 4. Read specific columns: `pd.read_csv('my_data.csv', usecols=['Name', 'Age'])`

```
[ ]: # YOUR PLAYGROUND - Work with files!

# Try reading with different options:
df = pd.read_csv('my_data.csv') # Add your own parameters!
```

```
print(df)

# Try reading only specific columns:
# df = pd.read_csv('my_data.csv', usecols=['Name', 'Age'])
# print(df)
```

1.16 Final Boss Level: Your Own Mini Project!

Now it's your turn to combine everything you've learned. Here's your challenge:

1.16.1 Challenge: Create Your Own Analysis

Your mission: 1. Create a DataFrame with at least 3 columns and 20 rows of data (make it about something you're interested in!) 2. Calculate some statistics (mean, max, min, etc.) 3. Filter the data based on some condition 4. Group the data and calculate aggregates 5. Create at least one visualization 6. Save your results to a CSV file

Ideas for your dataset: - Sports statistics (player names, scores, teams) - Book collection (titles, authors, ratings, pages) - Music playlist (songs, artists, duration, genre) - Daily expenses (date, category, amount, description) - Weather data (date, temperature, humidity, conditions) - Video games (title, platform, hours played, rating)

Use the cells below to build your project!

```
[ ]: # YOUR PROJECT - Step 1: Create your DataFrame

# Example structure to get you started:
# my_df = pd.DataFrame({
#     'column1': [...],
#     'column2': [...],
#     'column3': [...]
# })

# Your code here!
```

```
[ ]: # YOUR PROJECT - Step 2: Explore and analyze

# Your code here!
```

```
[ ]: # YOUR PROJECT - Step 3: Filter and group

# Your code here!
```

```
[ ]: # YOUR PROJECT - Step 4: Visualize

# Your code here!
```

```
[ ]: # YOUR PROJECT - Step 5: Save your work  
  
# Your code here!
```

1.17 Congratulations!

You've made it through the pandas learning adventure! Here's what you've learned:

- Creating Series and DataFrames
- Viewing and inspecting data
- Selecting data (columns, rows, conditions)
- Boolean indexing
- Statistical operations
- Handling missing data
- Grouping and aggregating
- Merging and joining
- String operations
- Pivot tables
- Data visualization
- Reading and writing files

1.17.1 Next Steps:

- Try analyzing real datasets (Kaggle has tons of free datasets!)
- Explore more advanced pandas features
- Combine pandas with other libraries like scikit-learn for machine learning
- Build a data dashboard with Streamlit or Plotly Dash

1.17.2 Resources:

- [Official Pandas Documentation](#)
- [Pandas Cheat Sheet](#)
- [Real Python Pandas Tutorials](#)

Remember: The best way to learn is by doing. Keep experimenting, keep breaking things, and keep building!

Happy Data Analyzing!